SC1001: Introduction to Computational Thinking

Week 11: Accumulators and Asymptotic Complexity

Jordan Boyd-Graber

Nanyang Technological University

Slides

Slides and updates will be posted at

`boydgraber.org/teaching/SC_1001/`

Overview

1. Compare recursive, tail-recursive, and iterative accumulation
2. Generate binary strings recursively
3. Add constraints to recursive generation
4. Practice asymptotic notation

Problem 1: List Length

Reimplement a version of Python's built-in `len` function for lists.

Write three versions:

- Recursive, without an accumulator
- Tail recursive, with an accumulator
- Iterative, with no recursion

Problem 2: Count Vowels

Write a function that counts the vowels in a string.

Write the same three versions:

- Recursive, without an accumulator
- Tail recursive, with an accumulator
- Iterative, with no recursion

Warm-Up: All Binary Strings

Before adding constraints, write a recursive function that returns all binary strings of length n.

For $n = 3$, the output should contain:

$$000, 001, 010, 011, 100, 101, 110, 111$$

Use a prefix argument or another way to remember the choices made so far.

Problem 3: Binary Strings Without 00

A binary string uses only `0` and `1`.

Write a recursive Python function that takes an integer $n \geq 1$ and returns all binary strings of length n without consecutive `0`s.

Start from the warm-up solution, then decide when the `0` branch is allowed.

Problem 4: Hamming Weight

The Hamming weight of a binary string is the number of 1s in it.

Write a recursive function that takes integers $n \geq 1$ and $c \geq 0$ and returns all binary strings of length n with Hamming weight at most c.

Problem 5: Big O

Give the big O notation for each worst-case operation count.

1. $n + 12 \log n$
2. 23
3. $n \log n$
4. $\log_5 n + \log_{10} n$
5. $\sqrt{n^{64}} + n^{32} + 994$